

Week 13, Lecture 25, HW and Q

Week 13, Lecture 25, Homework and Weekly Quiz with Solutions.

Week 13, Lecture 25: Homework and Weekly Quiz

Homework

Problem 1: Tensor shapes for DSP objects

For each object, state a sensible PyTorch shape and its tensor rank.

1. One mono audio clip containing $L = 16000$ samples.
2. A batch of $N = 32$ stereo clips, each containing $L = 16000$ samples, using the (N, C, L) convention.
3. A complex STFT of one mono clip with $F = 257$ frequency bins and $T = 101$ frames.
4. A batch of 32 magnitude spectrograms with one channel, 257 frequency bins, and 101 frames.

Solution

1. Shape $(16000,)$, rank 1.
 2. Shape $(32, 2, 16000)$, rank 3.
 3. Shape $(257, 101)$, rank 2, typically complex dtype.
 4. A natural image-like convention is $(32, 1, 257, 101)$, rank 4.
-

Problem 2: Broadcasting channel gains

A tensor x has shape $(24, 2, 8000)$. You want to multiply every left-channel sample by 0.5 and every right-channel sample by 1.2.

1. What shape should the gain tensor have to broadcast cleanly across batch and time?
2. Write one PyTorch expression that constructs the gain tensor.
3. What is the output shape?

Solution

Use shape

$(1, 2, 1)$.

For example:

```
g = torch.tensor([0.5, 1.2]).view(1, 2, 1)
y = x * g
```

The output shape remains

(24,2,8000).

Problem 3: Filterbank matrix multiplication

A power spectrogram is

$$\mathbf{P} \in \mathbb{R}^{257 \times 100},$$

and a 40-band filterbank is

$$\mathbf{W} \in \mathbb{R}^{40 \times 257}.$$

1. Write the matrix expression for filterbank energies.
2. State the result shape.
3. Explain what the two result dimensions mean.

Solution

$$\mathbf{E} = \mathbf{WP}.$$

Therefore,

$$\mathbf{E} \in \mathbb{R}^{40 \times 100}.$$

There are 40 filterbank energies for each of 100 time frames.

Problem 4: Chain rule by hand and with autograd

Let

$$u = 2\theta - 3,$$

and

$$L = (u + 1)^2.$$

1. Derive $dL/d\theta$.
2. Evaluate it at $\theta = 4$.
3. Give equivalent PyTorch code.

Solution

First,

$$\frac{dL}{du} = 2(u + 1), \quad \frac{du}{d\theta} = 2.$$

Thus,

$$\frac{dL}{d\theta} = 4(u + 1) = 4(2\theta - 2) = 8\theta - 8.$$

At $\theta = 4$,

$$\frac{dL}{d\theta} = 8(4) - 8 = 24.$$

```
theta = torch.tensor(4.0, requires_grad=True)
u = 2*theta - 3
L = (u + 1)**2
L.backward()
print(theta.grad)    # tensor(24.)
```

Problem 5: Why gradients accumulate

Run conceptually:

```
x = torch.tensor(3.0, requires_grad=True)
(x**2).backward()
(x**2).backward()
```

1. What is `x.grad` after the first call?
2. What is it after the second call?
3. Why do training loops call `optimizer.zero_grad()`?

Solution

For

$$L = x^2,$$

we have

$$\frac{dL}{dx} = 2x = 6$$

at $x = 3$.

After the first backward call, `x.grad = 6`. After the second, PyTorch adds another 6, so `x.grad = 12`. Optimizers normally clear gradients each iteration because a new minibatch should not unintentionally inherit the previous minibatch's gradient.

Problem 6: Learnable FIR gradient

For

$$\hat{y}[n] = \sum_{k=0}^{M-1} h[k]x[n-k]$$

and

$$L = \frac{1}{N} \sum_n (\hat{y}[n] - y[n])^2,$$

derive $\partial L / \partial h[r]$.

Solution

Define

$$e[n] = \hat{y}[n] - y[n].$$

Then

$$L = \frac{1}{N} \sum_n e^2[n].$$

By the chain rule,

$$\frac{\partial L}{\partial h[r]} = \frac{1}{N} \sum_n 2e[n] \frac{\partial \hat{y}[n]}{\partial h[r]}.$$

Since

$$\hat{y}[n] = \sum_k h[k]x[n-k],$$

we have

$$\frac{\partial \hat{y}[n]}{\partial h[r]} = x[n-r].$$

Therefore,

$$\boxed{\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n e[n]x[n-r]}.$$

This is a correlation-like gradient: the output error is correlated with delayed input samples.

Problem 7: Graph-breaking operations

Explain the difference between `torch.no_grad()` and `.detach()`. Give one appropriate use of each.

Solution

`torch.no_grad()` creates a context in which operations are executed without constructing autograd history. It is useful during inference or when generating a fixed target.

`.detach()` returns a tensor view of the same numerical values that is disconnected from the existing graph. It is useful when exporting or inspecting intermediate values without letting future operations backpropagate through the earlier graph.

Neither should be inserted into a path through which gradients are required.

Weekly Quiz: 10 minutes

Q1

A minibatch contains 16 mono clips, each 4000 samples long. Using (N, C, L) , what is the tensor shape?

Q2

A tensor has shape $(8, 2, 1000)$. What gain shape applies one independent gain to each channel and broadcasts over batch and time?

Q3

True or false: every PyTorch tensor automatically tracks gradients.

Q4

Let

$$L = (3\theta + 1)^2.$$

Find

$$\left. \frac{dL}{d\theta} \right|_{\theta=2}.$$

Q5

Why is reverse-mode autodiff especially appropriate when a model has many parameters but one scalar loss?

Q6

What is the most likely bug if a training loop calls `loss.backward()` every iteration but never clears parameter gradients?

Quiz Solutions

A1

(16,1,4000).

A2

(1,2,1).

A3

False. Gradient tracking is enabled for suitable floating/complex tensors when `requires_grad=True`, typically for learnable parameters.

A4

$$\frac{dL}{d\theta} = 2(3\theta + 1)(3) = 6(3\theta + 1).$$

At $\theta = 2$,

42.

A5

A single reverse pass reuses the computational graph to compute gradients of one scalar objective with respect to many upstream parameters efficiently.

A6

Gradients accumulate across iterations, so the optimizer uses the sum of current and previous gradients rather than only the intended current gradient.